



计组强化 P1: 数据的表示和运算



本讲目标：强化第二章大题与小题中整数和浮点数的表示、类型转换；以及整数的加减乘除运算、标志位生成；浮点数的加减运算解题能力。

一、整数的表示与类型转换

1. n bit 整数表示范围

- 无符号整数： $0 \sim 2^n - 1$
- 带符号整数（补码）： $-2^{n-1} \sim 2^{n-1} - 1$

手算判断溢出的朴素原理：先手算求数学真值，再检查是否超出相应类型的表示范围。

2. 补码快速转原码

- 符号位为 0：补码与原码相同，直接解释真值。
- 符号位为 1：从右向左找到第一个 1；该位及其右边保持不变；它左边的所有数值位取反；符号位保持为 1。
- 特别地：补码位串 $1000 \dots 00$ 表示 -2^{n-1} ，无法转换成同位宽原码，直接写真值。

3. 高频考点：不同类型整数变量之间转化，位宽和类型变化

- 一句话总结：先按“原有符号性”完成扩展/截断，再按“目的符号性”解释。
- 同宽转换：位串不变，只改变解释方式。
- 位宽变大
 - 原数是带符号数：先符号扩展（复制符号位），再按目的类型解释。
 - 原数是无符号数：先零扩展（高位补 0），再按目的类型解释。
- 位宽变小：截断，只保留低位，再按目的类型解释。



以下涉及“超范围值转带符号窄类型”的题目，均按 408 常用补码机器的“保留低位”模型作答；严格 C 标准下，结果可能与具体实现有关。

- 可能发生整数位宽变化的地方

层次	典型情况	示例
C 语言层	赋值、强制类型转换	<code>short → int</code> 、 <code>int → short</code>

层次	典型情况	示例
汇编语言层	指令指定符号扩展或零扩展	MOVSB、MOVZB
硬件层	短立即数、ALU 输入需要统一位宽	12 位立即数扩展为 32 位
硬件层	宽结果写回窄寄存器或只保留低位	只写回 2n 位乘积的低 n 位部分

eg: $PC + \text{偏}$ $\xrightarrow{12\text{位立即数(转移指令)}} \text{ALU or 加法器}$

- x86 汇编语言指令示例 (拓展, 不考)

```

mov ax, 0FF86H ; AX=FF86H, 0FF86H=FF86H, 前导 0 仅为汇编语法需要, 不影响数值
movsx ebx, ax ; 符号扩展, EBX=FFFFFF86H, 按补码解释为 -122
movzx eax, ax ; 零扩展, EAX=0000FF86H, 按无符号数解释为 65414
; 观察“宽数据的低位部分”。
mov eax, 12345678H ; EAX=12345678H, 此时低 16 位子寄存器 AX 自然为 5678H
; 后续若只读取 AX, 或将 AX 写入 16 位内存单元, 就相当于只保留低 16 位
; 注意: 不能写 mov ax, eax, 因为 mov 指令的两个操作数必须同宽

```

Handwritten notes:
 - 16bit 到 32bit
 - short 内存变量 $\Rightarrow$ mov 地址, AX (内存)
 - X mark under 'mov ax, eax'

扩展方式由 **MOVSB** 或 **MOVZB** 明确指定, 而不是由 EAX、EBX 本身决定, 机器并不知道原本存的是带符号 or 无符号数。得由编译器 or 汇编语言程序员按照正确逻辑使用对应的指令。

课堂练习 1 | 位宽和类型变化

假定 **short**、**unsigned short** 为 16 位, **int**、**unsigned int** 为 32 位。分别求最终机器数和真值。

```

// I. 位宽变大 (16→32)、类型不变
short a = -122;  $\Rightarrow$  FF86H  $\xrightarrow{32\text{位符号}} \text{FFFF FF86H}$   $\xrightarrow{\text{解得}} -122$ 
int b = a; // 带符号→带符号

unsigned short c = 0xFF86;  $\xrightarrow{32\text{位原码}} \text{0000 FF86H}$   $\xrightarrow{\text{目的数}} 65414$ 
unsigned int d = c; // 无符号→无符号

// II. 位宽变小 (32→16)、类型不变
int g = 0x00018009;  $\xrightarrow{\text{截断}} \text{0009H}$   $\xrightarrow{\text{真值}} 9$ 
short h = g; // 带符号→带符号

// III. 位宽不变 (16→16)、类型变化 (带符号/无符号互转)
short e = -1;  $\Rightarrow$  FFFFH  $\xrightarrow{\text{真值}} 65535$ 
unsigned short f = e; // 带符号→无符号

// IV. 位宽变大 (16→32)、类型也变 (带符号/无符号互转)
unsigned short p = 0xFF86;  $\xrightarrow{32\text{位原码}} \text{0000 FF86H}$   $\xrightarrow{\text{真值}} 65414$ 
int q = p; // 无符号→带符号

short r = -1;  $\Rightarrow$  FFFFH  $\xrightarrow{32\text{位原码}} \text{FFFF FFFFH}$   $\xrightarrow{\text{真值}} 2^{32}-1$ 
unsigned int s = r; // 带符号→无符号

// V. 位宽变小 (32→16)、类型也变 (带符号/无符号互转)

```

```

int t = -1;  $\Rightarrow$  FFFF FFFF  $\xrightarrow{16\text{bit}}$  FFFF  $\xrightarrow{8\text{bit}}$  65535
unsigned short u = t; // 带符号 $\rightarrow$ 无符号

unsigned int t2 = 65535;  $\Rightarrow$  0000 FFFF  $\xrightarrow{16\text{bit}}$  FFFF  $\xrightarrow{8\text{bit}}$  -1
short u2 = t2; // 无符号 $\rightarrow$ 带符号 (截断保留低16位, 再按补码解释)

```

课堂练习 2 | 指令与立即数扩展

某内存单元存放字节 86H。

1. 执行 `MOVSX EAX, byte ptr [x]` 后, EAX 是多少?
2. 执行 `MOVZX EBX, byte ptr [x]` 后, EBX 是多少?
3. 某 32 位指令包含 12 位补码立即数 F80H。该指令执行“寄存器值+立即数”的 ALU 加法, 立即数送入 ALU 前应扩展成什么机器数?

内存变量参与运算 (硬件本质): 从内存取到寄存器 $\rightarrow$ 必要时按“原符号性”做扩展到运算位宽 $\rightarrow$ ALU 做加减乘除 $\rightarrow$ 写回窄变量时只保留低位 (截断)。

课堂练习 3 | 整型提升与组合转换

假定 `short` 为 16 位, `int`、`unsigned int` 为 32 位。

```

short a = 30000;  $\xrightarrow{16\text{bit}}$   $\Rightarrow$  32bit  $\Rightarrow$  a+b  $\Rightarrow$  32bit  $\xrightarrow{16\text{bit}}$  30内存
int b = 10000;
short c = a + b;
16bit

unsigned short x = 65535;  $\xrightarrow{16\text{bit}}$   $\Rightarrow$  32bit  $\Rightarrow$  x+y  $\Rightarrow$  32bit (2)
int y = -1;
int z = x + y;

short m = -32767;
unsigned int n = m;

// 下面这个例子太难太偏了, 不重要, 当思维拓展即可
short s1 = -1;
unsigned int u1 = 1; 跳
unsigned int r1 = s1 + u1; // 提示: 先整型提升(short $\rightarrow$ int), 再以unsigned int参与运算

```

1. 写出加法 `a+b` 的完整结果以及 `c` 的机器数和真值。
2. 说明 `x` 参与加法前提升为什么类型, 并求 `z`。
3. 求 `n` 的机器数和无符号真值。
4. 拓展: 求 `r1` 的机器数和无符号真值。

课后真题加练

- 2009 年第 12 题: 整型提升与类型转换。
- 2012 年第 13 题: 无符号短整数扩展。

- 2015 年第 13 题：补码位串极值。
- 2016、2019 年第 13 题：signed 与 unsigned 转换。
- 2024 年第 12 题：截断与符号扩展。
- 2025 年第 12 题：负 `short` 转 `unsigned int`。

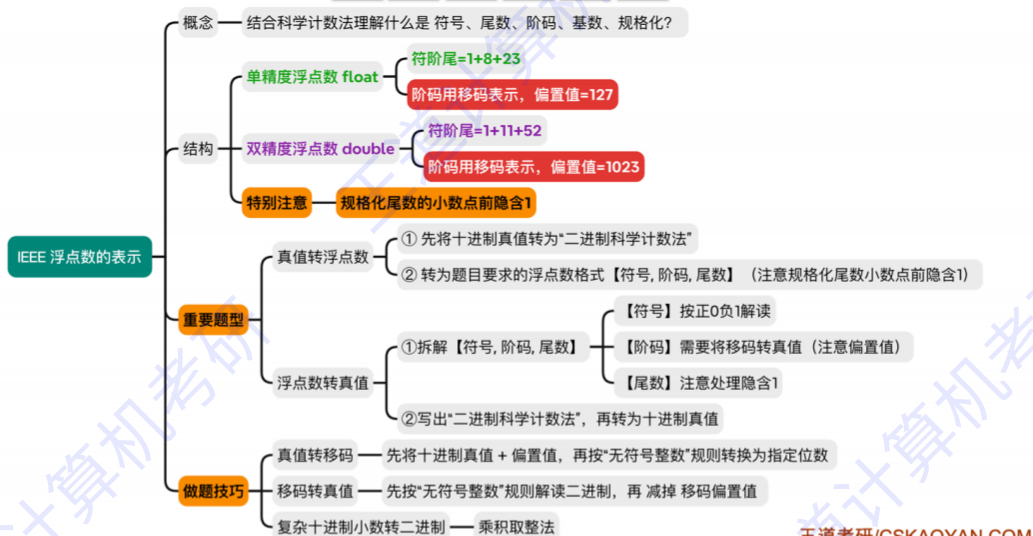
二、IEEE 754 浮点数（以 float 为例）的表示

舍入约定：本讲浮点转换和运算如未特别说明，均按“舍入到最近，恰好中点取偶”分析。

1. 浮点数结构与真值



知识回顾与重要考点



特殊状态的浮点数（阶码全0 或 全1）

值的类型	单精度 float (32位)				双精度 double (64位)			
	符号	阶码	尾数	值	符号	阶码	尾数	值
正零	0	全0	全0	+0	0	全0	全0	+0
负零	1	全0	全0	-0	1	全0	全0	-0
非规格化正数	0	全0	$f \neq 0$	$2^{-126}(0.f)$	0	全0	$f \neq 0$	$2^{-1022}(0.f)$
非规格化负数	1	全0	$f \neq 0$	$-2^{-126}(0.f)$	1	全0	$f \neq 0$	$-2^{-1022}(0.f)$
正无穷大	0	全1	全0	$+\infty$	0	全1	全0	$+\infty$
负无穷大	1	全1	全0	$-\infty$	1	全1	全0	$-\infty$
无定义数（非数）	0或1	全1	$f \neq 0$	NaN	0或1	全1	$f \neq 0$	NaN



阶码全为0、全为1 留作特殊用途，需要按照特殊的方式去解读真值



王道考研/CSKAOYAN.COM

$$\text{规格化浮点数真值} = (-1)^S \times (1.F)_2 \times 2^{E-127}$$

注意：规格化数具有隐藏的前导1，因此共有24位有效精度。

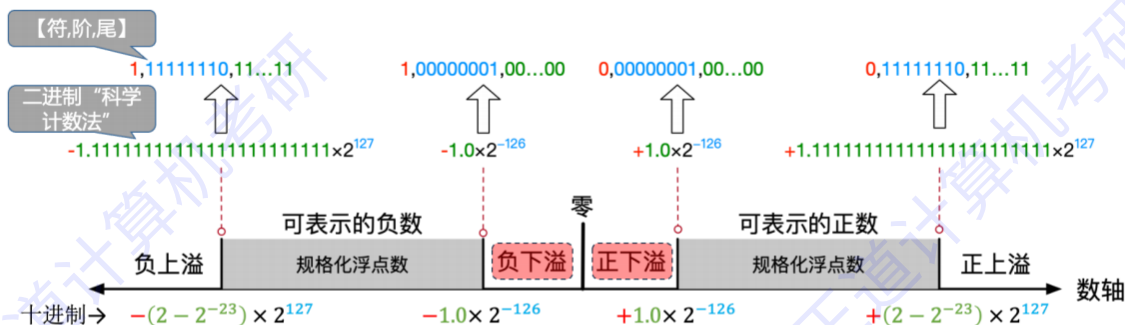
$$\text{非规格化浮点数真值} = (-1)^S \times (0.F)_2 \times 2^{-126}$$

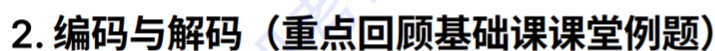
重要边界：

$$\text{最小正非规格化数} = 2^{-149}$$

$$\text{最小正规格化正数} = 2^{-126}$$

$$\text{最大有限正数} = (2 - 2^{-23})2^{127}$$





解码（二进制浮点数转十进制真值）：拆分 $S|E|F$ → 判断编码类别（规格/非规格/特殊值）→ 恢复 $1.F$ 或 $0.F$ → 求真实阶指数 → 计算真值。

判断十进制有理数转换成二进制小数后，能否在有限位后结束。技巧如下：

- 分解分母
- 分母只能由若干个2相乘得到，二进制小数才能有限结束
- 分母含3、5、7等其他因子，二进制小数会无限循环

一句话总结：确定真值的二进制能够有限位结束后，还要检查有效数字是否能装入 float 型的 24 位有效精度。例如： $2^{24} + 1$ 不能精确表示， $2^{24} + 2$ 可以精确表示。

4. 注意真值用浮点数能否装下: 检查范围

5. 浮点数大小比较

- 同为正规化数：正数按阶码、尾数顺序比较；负数按相反方向比较。
- 同为正非规格化数：尾数字段越大，真值越大；同为负非规格化数则相反。
- 任意正的规格化数大于任意正非规格化数；任意负规格化数小于任意负非规格化数。
- 一些特殊值的大小比较：
 - $+\infty$ 大于所有有限数， $-\infty$ 小于所有有限数，且 $+\infty > -\infty$ 。
 - $+\infty = +\infty$ ， $-\infty = -\infty$ ；但 $+\infty < +\infty$ 、 $-\infty < -\infty$ 均为假。
 - $+0 = -0$ 。
 - NaN 与其他数的大小比较为**无序**。无序不是“结果真假不确定”，而是两个操作数不属于 $<, =, >$ 中的任何一种关系；NaN 与自身也不相等。

课堂练习 4 | 单精度浮点数分类训练

下面 6 个 float 型变量的字段已经划分完毕。判断类型并解释真值或含义。

△先看阶码，注意全0/全1

变量	十六进制	S E F
a	00000001H	0 00000000 000000000000000000000001
b	007FFFFFH	0 00000000 111111111111111111111111
c	00800000H	0 00000001 000000000000000000000000
d	7F800000H	0 11111111 000000000000000000000000
e	FF800000H	1 11111111 000000000000000000000000
f	7FC00000H	0 11111111 100000000000000000000000

课堂练习 5 | 编码与解码训练

- 能否精确表示? $0.15 = \frac{3}{20} \rightarrow 2^2$
1. 将 -13.25 编码为 IEEE 754 单精度浮点数。
 2. 已知机器数 `0 | 10000010 | 010100000000000000000000`，求其真值。

课堂练习 6 | 0.125f 与 0.2f

```
float x = 0.125f; →  $\frac{1}{8} \rightarrow 2^3$ 
float y = 0.2f; →  $\frac{2}{10} = \frac{1}{5} \rightarrow$  不可表为  $2^n$  → 无限小数
```

已知 `x=3E000000H`，`y=3E4CCCCDH`

1. 哪一个可以精确表示?
2. `0.2f` 的实际存储值比 0.2 大还是小? ⚠ 坑, 大! → 舍入, 进位
3. 为什么将 `0.2f` 转为 `double` 后仍不能得到精确的 0.2? ← 留 24 位精度

课堂练习 7 | 精度、范围与大小比较

1. 判断 $2^{24} + 1$ 、 $2^{24} + 2$ 、 2^{128} 、 2^{-150} 能否成为有限 float，能否精确表示。
2. 比较 `007FFFFFH` 与 `00800000H` 的大小。
3. 比较 `807FFFFFH` 与 `80800000H` 的大小。

课后真题加练

- 2011 年第 13 题、2022 年第 14 题：单精度浮点数编码。
- 2012 年第 14 题：最大有限浮点数。
- 2013 年第 13 题、2023 年第 14 题、2025 年第 13 题：单精度浮点数解码。
- 2014 年第 14 题：浮点数比较。
- 2018 年第 14 题：最小正规格化数。
- 2021 年第 14 题：十进制数能否精确表示。

三、C 语言整型、浮点型跨类型转换

408 常考查 32 位 `int` 与 32 位 `float` 的转换：二者占用空间相同，但 `float` 只有 24 位有效精度。因此整数转 `float` 时通常不缺范围，却可能丢失精度。

1. 常见转换

转换	结论
<code>int</code> → <code>float</code>	可能丢失精度
<code>float</code> → <code>int</code>	范围内向 0 截断，超出目标整数范围时，标准 C 不保证结果
32 位 <code>int</code> → <code>double</code>	可以精确表示
<code>float</code> → <code>double</code>	已有 <code>float</code> 值可以精确扩大
<code>double</code> → <code>float</code>	可能舍入、上溢或下溢

2. x86 转换指令（拓展了解，考试不考）

C 转换	x86 常见指令
<code>int</code> → <code>float</code>	<code>CVTSI2SS</code>
<code>float</code> → <code>int</code> ，向 0 截断	<code>CVTSS2SI</code>
<code>float</code> → <code>double</code>	<code>CVTSS2SD</code>
<code>double</code> → <code>float</code>	<code>CVTSD2SS</code>

拓展了解（考试不考）：以下两条指令的具体写法仅作拓展，真题只要求掌握转换语义，不会考查汇编指令本身。

- `CVTSI2SS xmm0, eax`：把 `eax` 中的带符号整数转换为 `float`，写入 `xmm0` 低 32 位。例：`eax = 16777217` → `xmm0` 低 32 位为 `4B800000H`（即 `16777216.0f`，超出 24 位有效精度被舍入）。
- `CVTSS2SI eax, xmm0`：把 `xmm0` 低 32 位的 `float` 向 0 截断转换为 `int`，写入 `eax`。例：`xmm0` 中为 `123.75f` → `eax = 123`；为 `-123.75f` → `eax = -123`。

课堂练习 8 | 整数与浮点转换

```
int n = 16777217; // 01000001H, 即 2^24 + 1
float f = n;
double d = f;
```

```
int m = (int)f;
```

```
float a = 123.75f; 强转  $\Rightarrow$  int 123  
float b = -123.75f;  $\Rightarrow$  int -123
```

求 `f、d、m、(int)a、(int)b`。

课堂演示代码：示例 E。

课后真题加练

- 2009 年第 12 题、2012 年第 13 题、2016 年第 13 题、2019 年第 13 题、2024 年第 12 题、2025 年第 12 题：整数类型转换小题。
- 2010 年第 14 题：int、float、double 转换小题。
- 2017 年第 43 题：整数范围与浮点精度综合。

四、整数加减法、标志位与移位

1. 四个标志位

整数加减运算与这几个标志位的关系

标志	含义	生成逻辑?
CF	无符号加法产生进位，或无符号减法发生借位	
OF	带符号整数（补码）加减法溢出	
ZF	保留结果为 0	
SF	保留结果的最高位	

自己填（机器视角）

注意：SF 只是截断后结果的最高位；当 OF=1 时，不能根据 SF 判断数学真值的正负。

四个标志位（以 x86 为代表的 ISA：CF、OF、ZF、SF）之所以最常用于整数加减运算，核心原因是：加法/减法（以及比较 CMP，本质是做一次减法但不保存结果）会同时产生“结果值 + 进位/借位 + 符号信息”，这些信息正好能覆盖整数运算里最常见的判断：是否溢出、是否为 0、结果正负、无符号大小关系。因此后续很多条件分支（if/else）、循环判断都会直接依赖这四个标志位。

整数乘除运算与这几个标志位的关系

- x86 MUL/IMUL 主要规定 CF、OF，溢出时置为 1；ZF、SF 等通常未定义。
- x86 DIV/IDIV 的 CF、OF、ZF、SF 通常未定义。
- x86 除法溢出异常通常不能表述为“OF 置 1”。

浮点数运算与这几个标志位的关系 $\Rightarrow$ FPU

- 普通浮点加减不按整数加减规则使用 CF、OF、ZF、SF；浮点运算状态通常由专用状态寄存器记录。具体浮点比较指令是否更新部分标志位，由 ISA 规定。

2. CMP 与 SUB 指令的区别（x86）

$\hookrightarrow$ xxx 减法

在 if-else 语句中，我们常对比两个整数的大小，在机器看来，无符号整数、带符号整数没有任何区别。都是按一套逻辑、一套电路去做减法运算，并生成标志位。

SUB A,B: 计算A-B, 保存结果并更新标志位
CMP A,B: 计算A-B, 不保存结果, 只更新标志位

3. x86 条件转移（不用记，重点感受 if else 的底层原理）

注： $\wedge$ “且”表示两个条件必须同时成立； $\vee$ “或”表示至少一个条件成立。

比较关系	数学写法	真题选项式写法	x86
相等（通用）	$ZF = 1$	ZF=1	JE
不等（通用）	$ZF = 0$	ZF=0	JNE
无符号小于	$CF = 1$	CF=1	JB
无符号大于等于	$CF = 0$	CF=0	JAE
无符号小于等于	$CF = 1 \vee ZF = 1$	CF=1 或 ZF=1	JBE
无符号大于	$CF = 0 \wedge ZF = 0$	CF=0 且 ZF=0	JA
带符号小于	$SF \oplus OF = 1$	SF $\neq$ OF	JL
带符号大于等于	$SF \oplus OF = 0$	SF=OF	JGE
带符号小于等于	$ZF = 1 \vee (SF \oplus OF = 1)$	ZF=1 或 SF $\neq$ OF	JLE
带符号大于	$ZF = 0 \wedge (SF \oplus OF = 0)$	ZF=0 且 SF=OF	JG

一个示例：同一条 CMP，不同的 signed/unsigned 分支

```
int x = -1, y = 1;
if (x > y) {
    // signed 大于: -1 > 1 ?
}

unsigned int ux = 0xFFFFFFFFu, uy = 1u;
if (ux > uy) {
    // unsigned 大于: 4294967295 > 1 ✓
}
```

对应汇编示意，重点看条件转移指令，以及如何根据符号位判断跳转条件：

```
; x 与 y 放在 eax, ebx
cmp eax, ebx
jg L_signed_gt      ; signed: (ZF=0) 且 (SF=OF)

; ux 与 uy 放在 eax, ebx (位串相同，但解释为无符号)
cmp eax, ebx
ja L_unsigned_gt    ; unsigned: (CF=0) 且 (ZF=0)
```

4. MIPS 简要对照（不用记，了解即可，时间紧跳过不学也行）

MIPS 没有 x86 的 CF、OF、ZF、SF，但比较大小时仍要区分带符号数与无符号数。

指令	含义
<code>beq rs,rt,label</code>	<code>rs=rt</code> 时跳转
<code>bne rs,rt,label</code>	<code>rs≠rt</code> 时跳转
<code>slt rd,rs,rt</code>	按带符号数比较； <code>rs<rt</code> 时 <code>rd=1</code>
<code>sltu rd,rs,rt</code>	按无符号数比较； <code>rs<rt</code> 时 <code>rd=1</code>
<code>blt / bge</code>	按带符号数进行小于/大于等于判断
<code>bltu / bgeu</code>	按无符号数进行小于/大于等于判断

要点： `slt/sltu` 只比较、不跳转，把结果 0/1 写入 `rd`，可再配合 `beq/bne` 决定是否跳转；
`blt/bge/bltu/bgeu` 通常是伪指令。本页涉及的两个通用寄存器之间的六种比较关系，均可用 `slt/sltu` 配合 `beq/bne` 实现。

```
# 实现 if ($s1 < $s2) goto L (带符号)
slt $t0, $s1, $s2    # $t0 = ($s1 < $s2) ? 1 : 0, 只写结果不跳转
bne $t0, $zero, L     # $t0≠0, 即 rs<rt 成立时跳转

# 伪指令写法，与上面两条等价
blt $s1, $s2, L       # 汇编器展开: slt $at,$s1,$s2 + bne $at,$zero,L
```

六种比较关系的推导： MIPS 真实指令只需 `beq/bne` + `slt/sltu`，其余大小关系靠交换操作数、结果取反得到。

关系	推导思路	真实指令组合
<code>a=b</code> 、 <code>a≠b</code>	直接判断	<code>beq / bne a,b,L</code>
<code>a<b</code>	直接判断	<code>slt rd,a,b</code> ， <code>rd=1</code> 成立
<code>a>b</code>	交换：等价于 <code>b<a</code>	<code>slt rd,b,a</code> ， <code>rd=1</code> 成立
<code>a≥b</code>	取反：非(<code>a<b</code>)	<code>slt rd,a,b</code> ， <code>rd=0</code> 成立 (配 <code>beq rd,\$zero,L</code>)
<code>a≤b</code>	交换+取反：非(<code>b<a</code>)	<code>slt rd,b,a</code> ， <code>rd=0</code> 成立

RISC 与 CISC 风格对照： MIPS (RISC) 硬件只做 `beq/bne` + `slt`，其余关系全由软件推导，`bgt/ble` 等同为伪指令；x86 (CISC) 则提供 `JL/JGE/JLE/JG` 等整套条件转移指令，靠标志位组合区分 (见上一小节表格)。两种风格殊途同归。

5. 移位运算

操作	补入内容	与乘除法的关系
左移 k 位	低位补 0	未丢失有效位时等价于乘 2^k
逻辑右移 k 位	高位补 0	无符号整数等价于除 2^k 后向下取整
算术右移 k 位	高位补符号位	负数通常向负无穷取整，不一定等于 C 语言向 0 截断的除法

例如，-5 算术右移 1 位得到 -3，而 C 语言 `-5/2` 得到 -2。

操作	x86指令	MIPS指令
左移	<code>SHL/SAL</code>	<code>SLL</code>

$-5 \Rightarrow 1101 \Rightarrow 1011 \xrightarrow{\text{右移}} 1101 \times \Rightarrow 1011 \Rightarrow -3$
 原 补 补 原

操作	x86指令	MIPS指令
逻辑右移	SHR	SRL
算术右移	SAR	SRA

(3分) 对于 x86 `SHL/SAL`：CF 保存最后一个被移出最高位的比特；仅当移位次数为 1 时， $OF = CF \oplus$ 结果最高位。移位次数大于 1 时，OF 通常未定义。

整数移位指令不能直接实现浮点数乘除 2。若将 float 位串强行当作整数移位，得到的通常是另一个无关浮点数。

课堂练习 9 | 同一 CMP 的 signed/unsigned 分支

已知 32 位变量 `int x=-1`、`unsigned int y=1`，其机器数分别为 `FFFFFFFH`、`00000001H`。执行 `CMP x,y`，即计算 $x - y$ 。

1. 求结果及 CF、OF、ZF、SF。
2. 按带符号数比较时，`JL` 是否跳转？
3. 按无符号数比较时，`JB` 是否跳转？
4. C 表达式 `x<y` 应使用哪类条件转移？

课堂练习 10 | 移位

1. 8 位补码 -5 算术右移 1 位，结果是多少？是否等于 C 表达式 `-5/2`？坑，右移不一定等效 $\div 2^k$
2. 8 位 `40H` 在 x86 中左移 1 位，结果、CF、OF 分别是什么？(拓展 可不做)
3. `1.0f` 的位串为 `3F800000H`。若将该位串当作 32 位整数左移 1 位，是否得到浮点数 2.0？

课后真题加练

- 2011 年第 17 题：无符号条件转移。
- 2013 年第 14 题、2014 年第 13 题：补码移位与溢出。
- 2018 年第 16、19 题：移位、CF 与 OF。
- 2021 年第 13 题：signed/unsigned 大小比较。
- 2011 年第 43 题：ALU 与标志位综合。

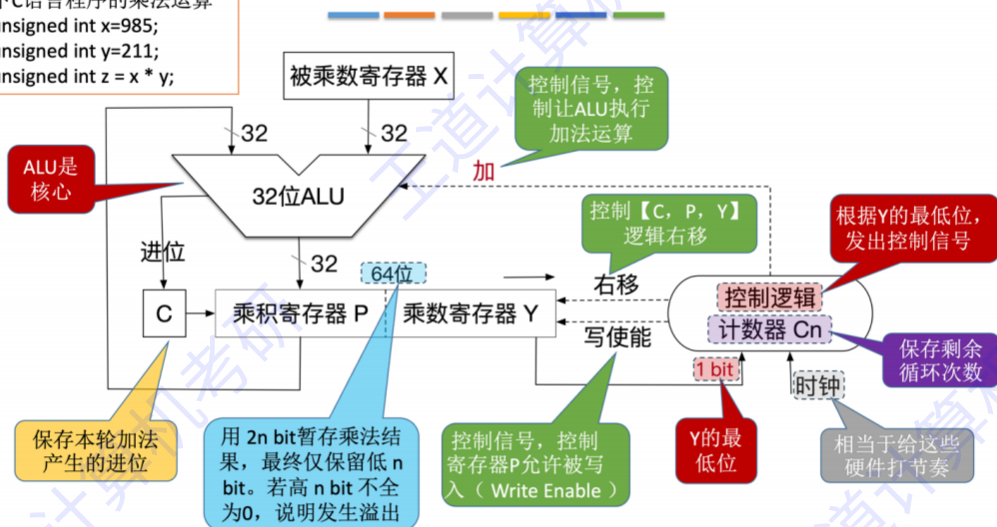
五、整数乘法

1. 三种实现方式

实现	核心思想	特点
纯软件	循环、移位、加法	无专用乘法硬件，速度较慢
加法器+移位	逐轮产生部分积	多周期、硬件适中
阵列乘法器	并行产生并累加部分积	速度快、硬件开销大

例如：可用该电路实现如下C语言程序的乘法运算
 unsigned int x=985;
 unsigned int y=211;
 unsigned int z = x * y;

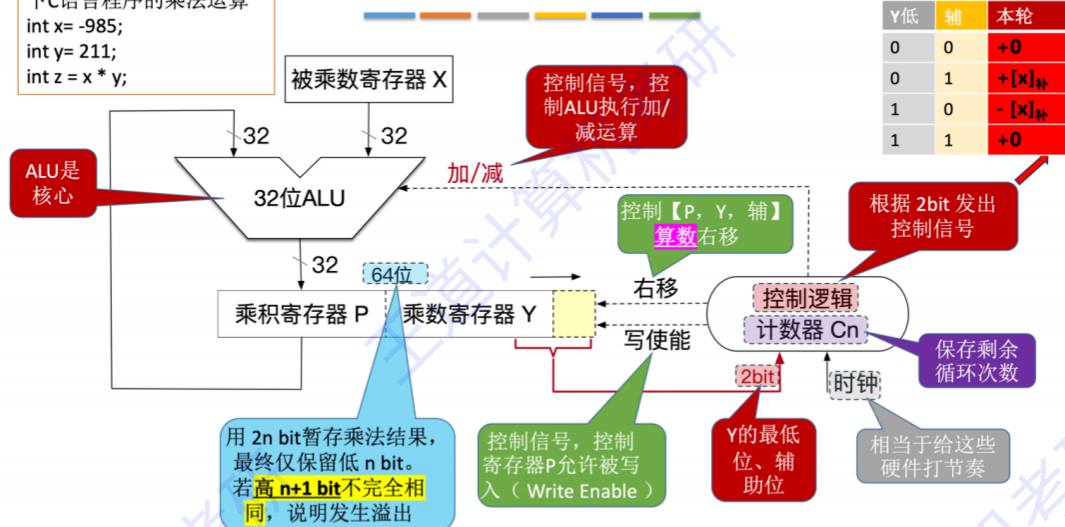
32 位无符号数乘法运算电路的结构



王道考研/CSKAOYAN.COM

例如：可用该电路实现如下C语言程序的乘法运算
 int x= -985;
 int y= 211;
 int z = x * y;

32 位带符号数乘法运算电路的结构



王道考研/CSKAOYAN.COM

2. 完整乘积与溢出

两个 n 位操作数的完整乘积最多需要 2n 位。只保留低 n 位时：

- 无符号乘法：高 n 位全 0，才不溢出。
- 带符号乘法：高 n 位必须是低 n 位符号位的完整扩展，才不溢出。

3. 复习建议

强化阶段优先掌握完整乘积、2n 位机器数、高低半部分、signed/unsigned 溢出判断和三种实现方式能简答即可。除非题目明确询问“第 k 轮寄存器内容”时才需逐轮模拟（这种考法过于变态不太可能出现）；考前后仍有余力且目标冲击高分的同学，可再尝试完整模拟。

课堂练习 11 | 完整乘积与溢出

1. 按带符号整数计算 16 位完整乘积，并判断只保留低 8 位是否溢出。
2. 按无符号整数计算 16 位完整乘积，并判断只保留低 8 位是否溢出。
3. 简述纯软件、加法器+移位、阵列乘法器如何完成该运算。

- 2010 年第 13 题：8 位补码乘法溢出。
- 2020 年第 43 题：软件乘法与乘法器。
- 2024 年第 15 题：乘法实现方式小題。

除数寄存器 Y

32

32

32位ALU

32

64位

余数寄存器 R

余数/商寄存器 Q

控制【R, Q】左移

左移

写使能

控制逻辑

计数器 Cn

时钟

保存剩余循环次数

控制信号，让ALU执行加法、减法运算

加/减

根据余数、除数的值，发出控制信号

【R】最终保存n bit 余数

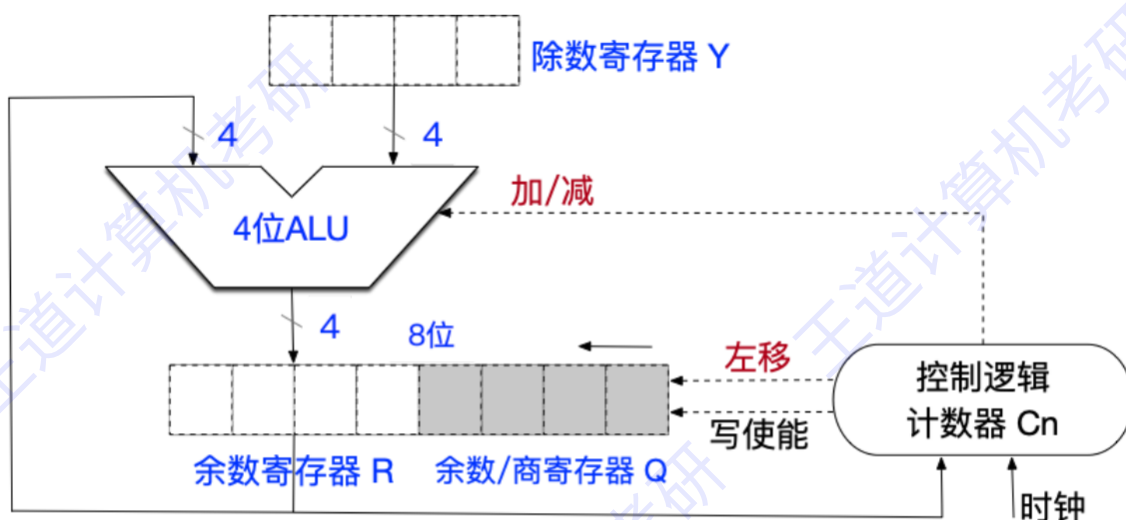
开始运算前，需要用【R, Q】2n bit 存放被除数，若被除数不足2n bit，需要进行位扩展

【Q】最终保存n bit 商

ALU是核心

浏览试卷

王道考研/CSKAOYAN.COM


$$x = qy + r, \quad |r| < |y|$$

一句话总结：**无符号除法 = 小学学的除法**（商向下取整、余数非负）；**带符号除法**需要额外约定“**商向 0 截断、余数与被除数同号**”。

2. 注意几种除法异常

- 无符号数、带符号数，都要注意：除数为 0。
- 带符号数特别注意：**绝对值最大负数 / -1** 导致商超出正数补码范围。
eg: 32bit int, a = -5, b = 2 ⇒ 初、最终 不要+数, 不进位 商 = -2 ⇒ 余数 = -1
- 单字长被除数 vs 双字长被除数的无符号除法：
 - 对于“**n 位无符号被除数 ÷ n 位非零无符号除数 → n 位商**”，商不可能溢出；除数为 0 仍然会触发异常。
 - 对于“**2n 位无符号被除数 R:Q ÷ n 位除数 Y → n 位商**”：**R < Y** 时商可装入 n 位，**R ≥ Y** 时商溢出。

3. 被除数扩展为双倍字长【R:Q】的规则：

无符号被除数：零扩展——Q=被除数，R 全 0（例：8 位 239 → R:Q=00EFH）；

带符号被除数（补码）：符号扩展——R 与被除数符号位一致，非负全 0、负数全 1（例：8 位 -17 → R:Q=FFE7H）；

点明这与第一章“按原有符号性扩展”规则一致，并标注这是 2025 年第 44 题“寄存器初值”的直接考点。

课堂练习 12 | 带符号与无符号除法

分别基于 8 位除法电路判断：

- 8 位带符号除法：-17/5, -128/-1, 100/0
- 8 位无符号除法：255/1, 100/0
- 16 位被除数、8 位商：R:Q=01F0H、Y=02H；R:Q=0100H、Y=01H

判断商、余数及异常情况。

课后真题加练

- 2025 年第 44 题：补码除法器、寄存器初值与异常。

七、浮点数加减

1. 基本过程

对阶

检查特殊值 → 对阶 → 尾数加减 → 规格化 → 舍入 → 再次规格化 → 检查阶码

对阶遵循“小阶向大阶对齐”，可能出现“大数吞小数”。对阶右移时保留 G、R、S 位；就近舍入、恰好中点取偶时，进位条件为 $G \wedge (R \vee S \vee L)$ ，其中 L 为保留部分最低位。

2. 注意！浮点数加减运算各阶段风险表：

阶段	风险或现象	float 型加减示例	结论
对阶	低位丢失、大数吞小数	$2^{100} + 1 = 2^{100}$	小操作数被移出有效精度

阶段	风险或现象	float 型加减示例	结论
尾数相加	尾数产生进位	$1.5f + 1.5f = 3.0f$	需要右规，但不自动上溢
尾数相减	有效数字大量抵消	$(1 + 2^{-23}) - 1 = 2^{-23}$	结果需要多次左规
右规	阶码加 1，可能上溢	<code>FLT_MAX+FLT_MAX</code>	阶码越界，得到 $+\infty$
左规	阶码减小，可能进入非规格化区	<code>00800000H-007FFFFFFFH=00000001H</code>	结果为最小正非规格化数
舍入	舍入进位可能再次右规	<code>3FFFFFFFH+33800000H=40000000H</code>	中点取偶后得到 2.0
最终封装	产生无穷、非规格化数或 0	<code>FLT_MAX+FLT_MAX</code> ；边界相减； <code>1.0f+(-1.0f)</code>	分别得到 $+\infty$ 、非规格化数和 0

3. 典型示例：两个最大有限 float 相加

设 $x = y = (2 - 2^{-23})2^{127}$ ：阶码相同 → 尾数相加产生进位 → 右规 → 阶码加 1 → 阶码超出范围 → 结果为 $+\infty$ 。

课堂练习 13 | 舍入与大数吞小数

每一步均舍入为 float 型：

1. 求 $1.0 + 2^{-24}$ 。
2. 比较 $A = (1.0 + 2^{-24}) + 2^{-24}$ 与 $B = 1.0 + (2^{-24} + 2^{-24})$ 。
3. 求 $2^{100} + 1$ ，并说明发生了什么现象。

课堂练习 14 | 阶段风险

判断：

1. 对阶会直接使最终阶码上溢。
2. 右规可能导致阶码上溢。
3. 左规可能导致阶码下溢。
4. 尾数产生进位时，浮点结果必然上溢。
5. 舍入可能使尾数进位，并间接导致阶码上溢。
6. `FLT_MAX+FLT_MAX` 的结果是什么？

课后真题加练

- 2010 年第 14 题：大数吞小数。
- 2015 年第 14 题：浮点加减各阶段风险。
- 2017 年第 43 题：精度、舍入与上溢综合。